Table of contents

Software Modeling and Verification

Introduction

The central question in this field is: **is my computing system, program, or software correct?**

This concern is critical at all scales:

- **Large-scale systems**: Internet, distributed systems
- **Embedded systems**: IoT, critical systems, automation

Verification addresses several fundamental aspects:

- **Algorithms** → their implementation as programs
- **Data types** → correct representation and manipulation
- **Protocols** → communication and coordination between components

---

Part 1: Basic concepts of modeling and verification

Program and specifications

Definition of a program

A program is initially a **sequence of characters** that must follow syntactic rules checked by a compiler.

Caution: A compiler only checks **syntax**, not semantic correctness!

From this sequence of characters, we build a **program model** – a mathematical abstraction that represents the program's semantics (its meaning).

Semantics: Meaning of syntactic constructs. Example: `x++` has the semantics "x is incremented by 1".

Formal models are **mathematical objects** that allow reasoning about program behavior in a systematic and, to some extent, automated way. This is particularly useful for complex systems where intuition may fail.

Functional and non-functional properties

Non-functional properties

These properties concern **quality of service** rather than specific computation:

- Execution time, performance
- Ergonomics, usability
- Security properties
- Resource consumption (memory, energy)

Functional properties

These properties specify **what the program is supposed to compute**.

Example: `qsort(T)` must return an array `T'` that is a sorted permutation of `T`.

Formal specifications

Specifications can be expressed in various ways:

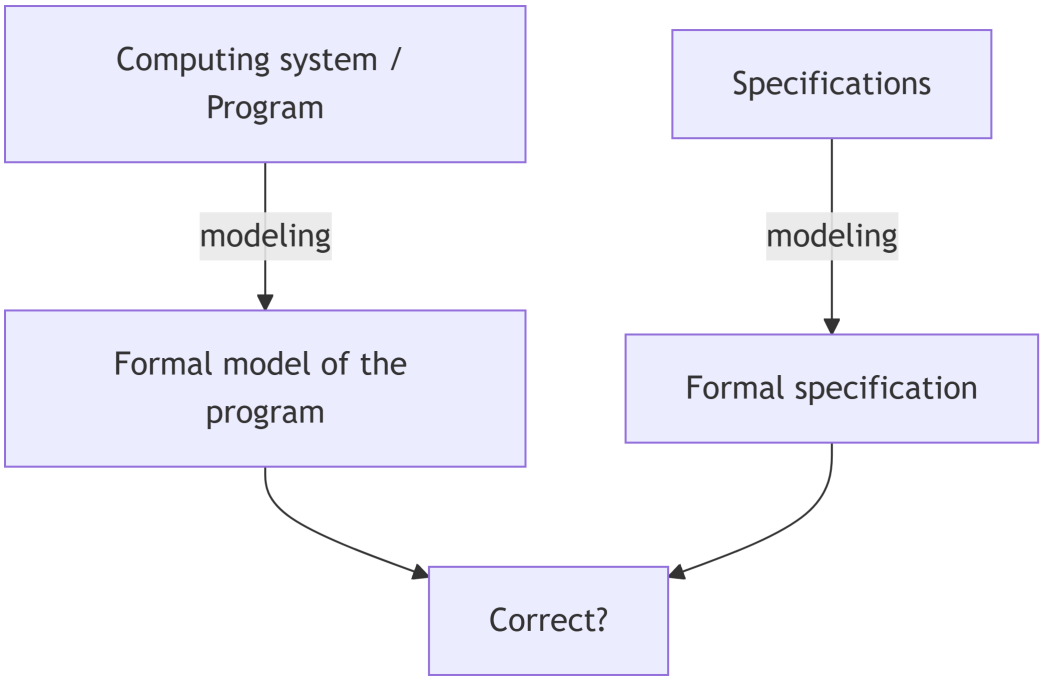
1. **Natural language**: Written descriptions, often ambiguous
2. **Program annotations**: Comments in the code
3. **Assertions**: Defensive programming constructs that detect errors at runtime
4. **Unit tests**: Specifications by examples
5. **Formal specifications**: Logical predicates with precise mathematical meaning

Example of a formal specification for a sorted array:

$$\forall i \in \{0, \dots, n-2\}, t[i] \leq t[i+1]$$

The verification problem

We want to determine if a program satisfies its specifications. This can be visualized as:



Definition: A program **satisfies** (or verifies) a specification, denoted `program |= specification`, if for all possible executions, the program's behavior conforms to the specification.

Verification methods

Several approaches exist to answer the correctness question:

Deductive Reasoning

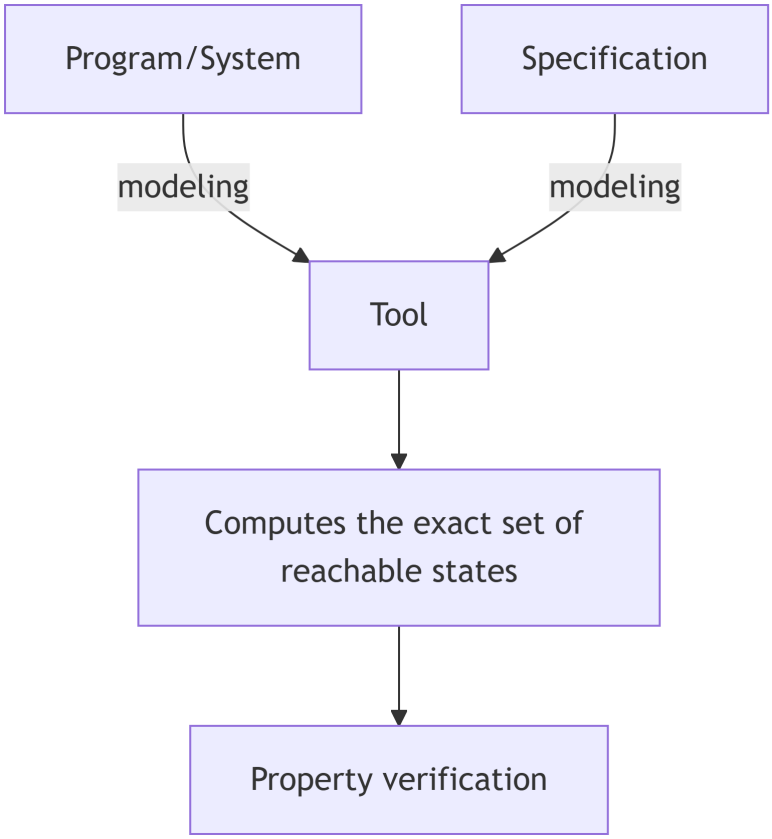
Uses mathematical logic to prove that a program satisfies its specification. **Hoare logic** is a fundamental example.

Abstract Interpretation

A technique that computes a **over-approximation** of the program's possible behaviors. If the over-approximation satisfies the property, then the program does too.

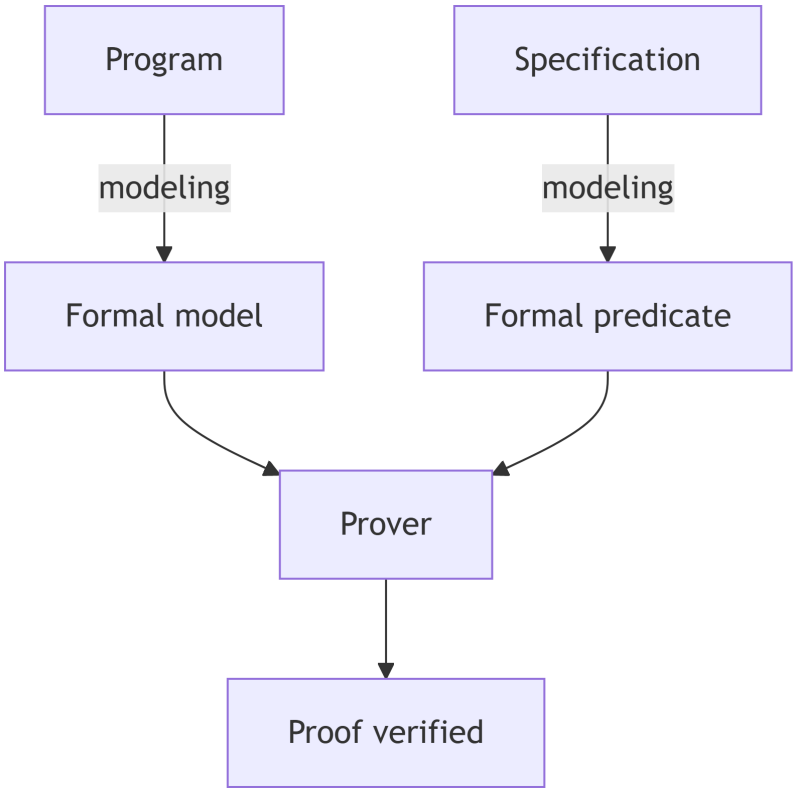
Model Checking

Automatically checks that a finite model of a system satisfies a formal property. Computes the **exact set** of reachable states (within computability limits).



Assisted Proof

Combines human assistance with automatic verification. The user guides the proof, and the machine verifies each step.



Part 2: Deductive reasoning and Hoare logic

Introduction to Hoare logic

Hoare logic (due to C.A.R. Hoare, 1969) is a formal system for reasoning about program correctness. It allows mathematically deriving that a program satisfies its specification.

Central idea: Associate with each program instruction how it transforms the properties of variables.

Operational semantics

Before introducing Hoare logic, we must formally define the meaning of programs. **Operational semantics** describes how the memory state evolves during execution.

Formal definitions

Memory state: Denoted μ, it is a function that associates each variable with its value.

$$\mu : Var \longrightarrow \mathbb{Z} \cup \{tt, ff\}$$

where Var is the set of program variables, $\mathbb{Z}$ the integers, and $\{tt, ff\}$ the booleans.

State update: $\mu[x \leftarrow c]$ denotes the state obtained by modifying μ so that x equals c, with other variables unchanged.

Expression evaluation: $Val(e, \mu)$ returns the value of expression e in state μ.

Instruction execution: $Mem(c, \mu)$ returns the new memory state after executing instruction c from state μ.

Example: $Mem(y := 0, \mu) = \mu[y \leftarrow 0]$

Hoare triples and partial correctness

Definition of triples

A **Hoare triple** has the form $\{P\}\ c\ \{Q\}$ where:

- P is a **precondition** (logical predicate on variables before execution)
- c is an instruction or sequence of instructions
- Q is a **postcondition** (logical predicate on variables after execution)

Meaning: If the memory state satisfies P before executing c, and if the execution of c terminates, then the resulting memory state will satisfy Q.

Validity of a triple

A triple $\{P\}\ c\ \{Q\}$ is **valid** if and only if:

$$\forall \mu, \mu \vDash P \Longrightarrow Mem(c, \mu) \text{ is defined and } Mem(c, \mu) \vDash Q$$

where $\mu \vDash P$ means "μ satisfies predicate P".

Partial correctness

The above definition assumes that $Mem(c, \mu)$ is defined, meaning that the execution of c **terminates**. A valid triple therefore guarantees that **if** the program terminates, then the postcondition is satisfied. This is called **partial correctness**.

Important note: The triple $\{false\}\ c\ \{Q\}$ is always valid (because the premise is always false). It is said to be **vacuously true**.

Weakest precondition (WP) calculation

The **Weakest Precondition** (WP) principle is to calculate, for an instruction c and a postcondition Q, the weakest condition on the initial state that guarantees that after executing c, Q will be true.

Recursive definition of WP

For basic instructions:

1. **Assignment**: $WP(x := t, Q) = Q[x \leftarrow t]$ where $Q[x \leftarrow t]$ is Q in which each free occurrence of x is replaced by t.
2. **Sequence**: $WP(c_1; c_2, Q) = WP(c_1, WP(c_2, Q))$
3. **Conditional**:

$$WP(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) = (b \Rightarrow WP(c_1, Q)) \wedge (\neg b \Rightarrow WP(c_2, Q))$$

Fundamental theorem for loop-free programs

For a program c without loops or function calls, and for formulas P and Q:

$$\{P\}\ c\ \{Q\} \text{ is valid } \iff P \Rightarrow WP(c, Q) \text{ is valid}$$

In other words, P must imply the weakest precondition of c for Q.

Loop handling: invariants

For loops, WP calculation is not directly possible (this would solve the halting problem, which is undecidable). The user must provide a **loop invariant**.

Definition of an invariant

Consider the loop: `while b do c done`

A predicate I is an **invariant** of this loop if and only if:

$$\{I \wedge b\}\ c\ \{I\} \text{ is a valid triple}$$

Intuition: If I is true before entering the loop body, and the loop condition b is true, then after executing the body, I remains true.

WP with invariant for a loop

Given an invariant I for the loop `while b do c done`, we define:

$$WP(\text{while } b \text{ do } c \text{ done}, Q) = I$$

provided that:

1. I is an invariant: $\{I \wedge b\}\ c\ \{I\}$ valid
2. Upon loop exit, I and the negation of the condition imply Q: $I \wedge \neg b \Rightarrow Q$

Practical example: Function `add(a, b)` with precondition $b \geq 0$ and postcondition $res = a + b$

```
add(a, b) =
  res := a;
  aux := b;
  while aux > 0 do
    res := res + 1;
    aux := aux - 1;
  done;
  res
```

Proposed invariant: $I = (res + aux = a + b) \wedge (aux \geq 0)$

We need to verify:

1. $\{I \wedge aux > 0\}\ res := res + 1; aux := aux - 1\ \{I\}$
2. $I \wedge \neg(aux > 0) \Rightarrow res = a + b$

The WP calculation for the loop body gives:

$$WP(res := res + 1; aux := aux - 1, I) = I[aux \leftarrow aux - 1][res \leftarrow res + 1]$$

Which simplifies to $(res + 1) + (aux - 1) = a + b \wedge aux - 1 \geq 0$, equivalent to $res + aux = a + b \wedge aux \geq 1$.

Now $I \wedge aux > 0$ implies $res + aux = a + b \wedge aux \geq 0 \wedge aux > 0$, so $aux \geq 1$, which indeed implies $WP(...)$.

For the exit condition: $I \wedge \neg(aux > 0)$ implies $res + aux = a + b \wedge aux \geq 0 \wedge aux \leq 0$, so $aux = 0$, hence $res = a + b$.

Thus, I is indeed a valid invariant.

Function call handling

Function contract

A function f can be associated with a **contract** $\{\phi\}\ f\ \{\psi\}$ where:

- ϕ are the preconditions on parameters
- ψ are the postconditions linking parameters and the return value

WP for a function call

For a call `a := f(b)` in a context where f has the contract $\{\phi\}\ f\ \{\psi\}$, we have:

$$WP(a := f(b), Q) = \phi[x \leftarrow b] \wedge \forall v, (\psi[x \leftarrow b, res \leftarrow v] \Rightarrow Q[a \leftarrow v])$$

where x is f's formal parameter, res its return value, and v a fresh variable.

In practice, a simplified version is often used:

$$WP(a := f(b), Q) = \phi[x \leftarrow b] \wedge (\psi[x \leftarrow b, res \leftarrow f(b)] \Rightarrow Q[a \leftarrow f(b)])$$

Example: Recursive function `sum(n)` that computes $\sum_{i=1}^{n} i$

```
sum(n) =
  requires { n >= 0 }
  ensures { res = Σ_{i=1}^n i }
  if n = 0 then
    res := 0
  else
    res := n + sum(n - 1)
  endif;
  res
```

To verify the contract, we need to compute WP of the body with the postcondition.

Let $\phi(n) = n \geq 0$ and $\psi(n, res) = res = \sum_{i=1}^{n} i$.

For the case `n = 0`: $WP(res := 0, \psi) = (0 = \sum_{i=1}^{0} i)$ which is true since $\sum_{i=1}^{0} i = 0$.

For the case `n  0`: $WP(res := n + sum(n-1), \psi)$

This becomes: $\phi(n-1)$ (precondition of the call) and $\psi(n-1, sum(n-1)) \Rightarrow \psi(n, n+sum(n-1))$.

Which gives: $(n-1 \geq 0) \wedge (sum(n-1) = \sum_{i=1}^{n-1} i) \Rightarrow (n + sum(n-1) = \sum_{i=1}^{n} i)$.

Since $\sum_{i=1}^{n} i = n + \sum_{i=1}^{n-1} i$, the implication is true.

Complete verification also requires showing that $n \geq 0 \Rightarrow n - 1 \geq 0$ when $n \neq 0$, which is the case.

Total correctness (variant)

Partial correctness does not guarantee **termination**. To obtain **total correctness**, we must add a **variant**.

Definition of a variant

A **variant** is an integer expression V over program variables such that:

1. $V \geq 0$ before each loop iteration or recursive call
2. V strictly decreases at each iteration/call

For loops: For `while b do c done` with invariant I, we require the existence of a variant V such that:

- $I \wedge b \Rightarrow V \geq 0$
- $\{I \wedge b \wedge V = v_0\}\ c\ \{V < v_0\}$ (where v_0 is a logical variable)

For recursive calls: For a function $f(x)$ with precondition $\phi(x)$, we require a variant $V(x)$ such that:

- $\phi(x) \Rightarrow V(x) \geq 0$
- For any call $f(y)$ in f's body, $\phi(x) \Rightarrow V(y) < V(x)$

Example: For the function `sum(n)`, the variant can be n itself.

We have: $n \geq 0 \Rightarrow n \geq 0$ (obvious) And for the recursive call `sum(n-1)`: $n \geq 0 \wedge n \neq 0 \Rightarrow n - 1 < n$ (true)

Thus, termination is guaranteed.

Correspondence theorem between operational and axiomatic semantics

This fundamental theorem establishes the equivalence between the operational definition of a Hoare triple's validity and its characterization by WP.

Statement of the theorem

For any program c (with or without loops, with invariant and variant handling) and for all formulas P and Q:

$$\{P\}\ c\ \{Q\} \text{ is valid (according to operational semantics)} \iff P \Rightarrow WP(c, Q) \text{ is valid}$$

where WP is extended to loops with invariants as described earlier.

Proof idea

The proof proceeds by **structural induction** on the program c. We present the key cases:

Assignment case

For $c = x := t$, we must show:

$$\{P\}\ x := t\ \{Q\} \text{ valid } \iff P \Rightarrow Q[x \leftarrow t] \text{ valid}$$

Proof: By operational definition, $\{P\}\ x := t\ \{Q\}$ is valid iff for all μ, if $\mu \vDash P$, then $Mem(x := t, \mu) = \mu[x \leftarrow val(t, \mu)] \vDash Q$.

Now, by a fundamental semantic property: $\mu \vDash Q[x \leftarrow t]$ iff $\mu[x \leftarrow val(t, \mu)] \vDash Q$.

So the condition is equivalent to: if $\mu \vDash P$, then $\mu \vDash Q[x \leftarrow t]$, i.e., $P \Rightarrow Q[x \leftarrow t]$.

Sequence case

For $c = c_1; c_2$, assuming the induction hypothesis for c_1 and c_2:

$\{P\}\ c_1; c_2\ \{Q\}$ valid
iff there exists R such that $\{P\}\ c_1\ \{R\}$ and $\{R\}\ c_2\ \{Q\}$ valid
iff (by IH) $P \Rightarrow WP(c_1, R)$ and $R \Rightarrow WP(c_2, Q)$
iff $P \Rightarrow WP(c_1, R)$ and $R \Rightarrow WP(c_2, Q)$ for some R

Now $WP(c_1; c_2, Q) = WP(c_1, WP(c_2, Q))$ is the weakest such R.

Therefore, this is equivalent to $P \Rightarrow WP(c_1, WP(c_2, Q)) = WP(c_1; c_2, Q)$.

Loop case

For $c =$ while b do c' done with invariant I:

The operational validity of $\{P\}\ c\ \{Q\}$ requires that for all μ with $\mu \vDash P$, the loop execution terminates in a state μ' such that $\mu' \vDash Q$.

By induction on the number of iterations and using the fact that I is preserved by each iteration, we show that this is equivalent to:

1. $P \Rightarrow I$ (the invariant is initially true)
2. $I \wedge b \Rightarrow WP(c', I)$ (the invariant is preserved)
3. $I \wedge \neg b \Rightarrow Q$ (the postcondition is satisfied upon exit)

This is exactly the definition of $P \Rightarrow WP(c, Q)$ for a loop, where $WP(c, Q) = I$ under conditions 2 and 3.

Thus, the two definitions coincide, establishing the correspondence between operational and axiomatic semantics through WP calculation.

This course has presented the foundations of software modeling and verification, focusing particularly on deductive reasoning and Hoare logic. These formal techniques allow rigorously establishing program correctness, thus complementing empirical methods like testing.